4. Pregunta 1 – Página web estática de Decroly

El enunciado pide una sola página, con una imagen y temática relacionada con Decroly y sus enseñanzas. La solución incluye texto sobre aprendizaje activo, observación, asociación y expresión, además de una ilustración SVG guardada en el propio proyecto.

4.1. Archivos de la página

```
src/main/webapp/
■■■ index.html
■■■ css/
■   ■■■ estilos.css
■■■ img/
    ■■■ decroly.svg
```

4.2. Cómo funciona el HTML

index.html define la estructura y el contenido. La etiqueta link enlaza el CSS; la etiqueta img muestra la ilustración local; el enlace con href="contador" será el acceso al servlet de la pregunta 3.

```
<head>
    <meta charset="UTF-8">
    <title>Decroly | Educación con valores</title>
    <link rel="stylesheet" href="css/estilos.css">
</head>
<body>
    <header class="barra">
        <a class="marca" href="index.html">Decroly</a>
    </header>
    <main>
        <section class="hero">
            <h1>Aprender para la vida.</h1>
            <p>Una educación activa que despierta la curiosidad.</p>
            <img src="img/decroly.svg" alt="Ilustración educativa">
        </section>
    </main>
</body>
```

4.3. Para qué sirve el CSS

estilos.css controla colores, tamaños, espacios, tarjetas y adaptación a pantallas pequeñas. No cambia la lógica del servidor; solo mejora la presentación visual.

```
body {
    margin: 0;
    font-family: Arial, sans-serif;
    color: #172554;
    background: #f4f8ff;
}
.hero {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 36px;
    padding: 65px 7%;
    background: #eaf4ff;
}
.boton {
    display: inline-block;
    padding: 13px 20px;
    color: white;
    background: #126bd0;
    border-radius: 9px;
    text-decoration: none;
}
```

5. Desplegar la web en Tomcat desde Eclipse

1. Abre la vista Servers.

2. Botón derecho sobre Tomcat v9.0 Server at localhost → Add and Remove…

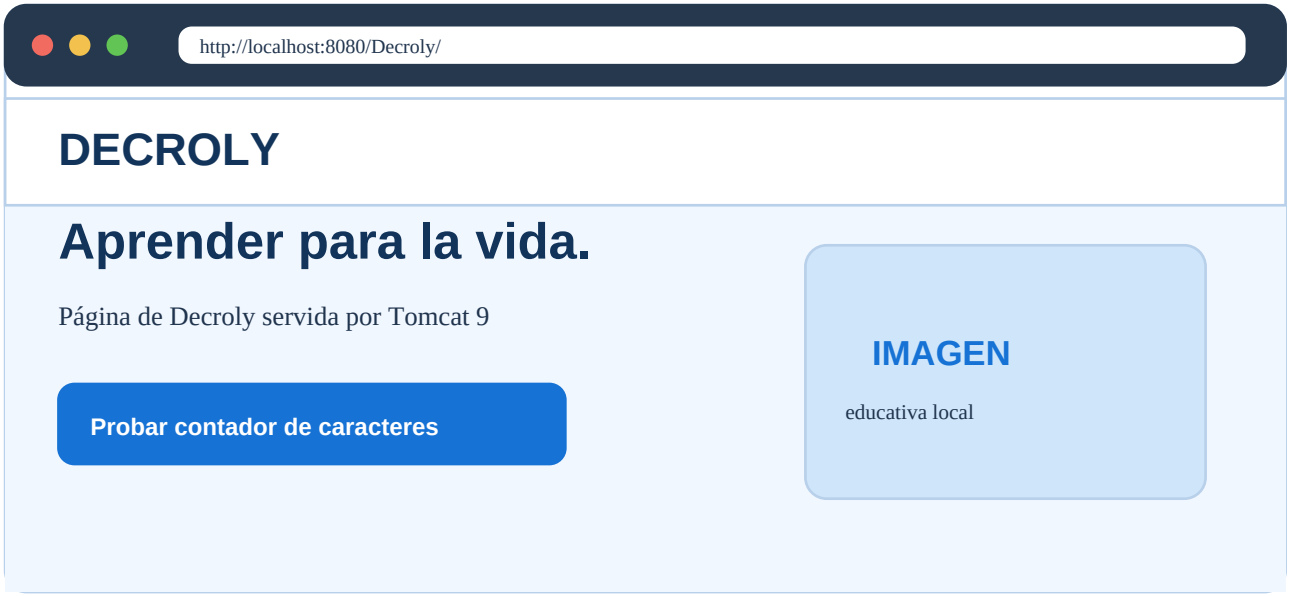
3. En Available selecciona Decroly y pulsa Add para pasarlo a Configured.

4. Pulsa Finish.

5. Botón derecho sobre Tomcat → Start. Espera a que el estado cambie a Started.

6. Abre **http://localhost:8080/Decroly/**. Si el contexto configurado es Decroly, debe aparecer la página.

Esquema de la página desplegada



Esquema ilustrativo, no captura real de Eclipse ni de tu navegador.

5.1. Qué carpeta mostrar en la evidencia

El enunciado pide una captura que muestre la carpeta donde se han desplegado HTML, CSS e imagen. En Eclipse puedes mostrar el proyecto y su estructura en Project Explorer; también puedes abrir la ubicación de despliegue del servidor desde la configuración de Tomcat. La ruta física concreta depende de cómo esté configurado Eclipse, así que no asumas que siempre será la carpeta webapps de la instalación original.

5.2. Texto que puedes adaptar para explicar el despliegue

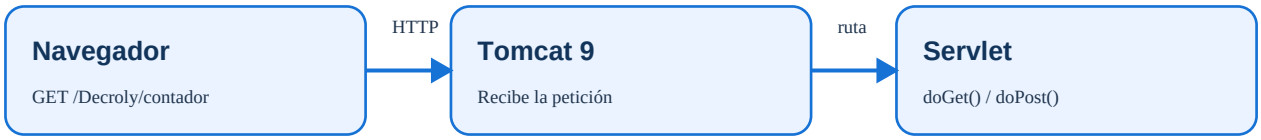
«He creado una página HTML con contenido relacionado con Decroly y una hoja de estilos CSS. He guardado la imagen dentro de la carpeta img para que la aplicación pueda cargarla desde una ruta relativa. Después he importado el proyecto en Eclipse, he añadido el proyecto al servidor Tomcat 9 desde Add and Remove y he iniciado el servidor. Finalmente he accedido a la dirección local para comprobar que la página se muestra correctamente».

6. Pregunta 2 – Crear el contador de caracteres

La aplicación debe aceptar un texto y devolver el número de caracteres. Además, el examen pide mostrar cuántos caracteres faltan para llegar a 280, que es el límite indicado en el enunciado.

6.1. ¿Qué es un servlet?

Un servlet es una clase Java que se ejecuta dentro de un contenedor web como Tomcat. Recibe peticiones HTTP y puede generar una respuesta HTML. En este ejercicio, el navegador envía el texto al servlet y el servlet calcula el resultado.



El servlet genera una respuesta HTML que Tomcat devuelve al navegador.

Flujo simplificado: el navegador envía una petición; Tomcat encuentra la ruta del servlet; el servlet procesa los datos y devuelve HTML.

6.2. Cómo se registra la ruta del servlet

```
@WebServlet("/contador")
public class ContadorServlet extends HttpServlet {
    // métodos doGet() y doPost()
}
```

La anotación @WebServlet("/contador") indica que el servlet responde en la ruta /contador dentro de la aplicación. Como el contexto es Decroly, la URL completa será http://localhost:8080/Decroly/contador.

6.3. Diferencia entre doGet y doPost en este ejercicio

• **doGet:** se ejecuta cuando se abre la dirección del contador. Muestra el formulario HTML.

• **doPost:** se ejecuta al enviar el formulario. Recoge el texto, calcula su longitud y construye la página de resultado.

6.4. Formulario que introduce el texto

```
<form method="post" action="contador">
    <label for="texto">Introduce tu texto:</label>
    <textarea id="texto" name="texto" required></textarea>
    <button type="submit">Contar caracteres</button>
</form>
```

El atributo name="texto" es importante: permite recuperar el valor en Java mediante request.getParameter("texto"). El método post envía los datos al servidor en el cuerpo de la petición.

7. Programar la lógica del servlet

7.1. Recibir el texto

```
request.setCharacterEncoding("UTF-8");
String texto = request.getParameter("texto");

if (texto == null) {
    texto = "";
}
```

request.setCharacterEncoding("UTF-8") permite interpretar correctamente caracteres acentuados y signos que llegan desde el formulario. getParameter("texto") obtiene el campo que tiene name="texto". La comprobación evita trabajar con null si el parámetro no llega.

7.2. Contar caracteres y calcular los que faltan

```
int caracteres = texto.length();
int faltan = Math.max(0, 280 - caracteres);
```

length() devuelve la longitud de la cadena. Si el texto tiene 100 caracteres, faltan 180; si tiene 280, faltan 0. Math.max(0, ...) impide que el resultado de «faltan» sea negativo cuando el texto supera el límite.

7.3. Mostrar el resultado

```
out.println("<h1>Resultado del contador</h1>");
out.println("<p>Tu texto tiene " + caracteres
    + " caracteres.</p>");

if (caracteres <= 280) {
    out.println("<p>Te faltan " + faltan
        + " caracteres para llegar a 280.</p>");
} else {
    out.println("<p>Has superado el límite en "
        + (caracteres - 280) + " caracteres.</p>");
}
```

Si el texto supera 280, la solución indica también cuánto se ha excedido. Es una mejora para que el resultado no resulte confuso.

7.4. Por qué escapamos el texto que se vuelve a mostrar

Si el usuario introduce signos como < o >, no conviene insertarlos directamente como HTML. El método escaparHtml convierte esos caracteres en entidades HTML para que se muestren como texto. Esto evita que una entrada se interprete como una etiqueta HTML.

```
private String escaparHtml(String texto) {
    return texto.replace("&", "&amp;")
                .replace("<", "&lt;")
                .replace(">", "&gt;")
                .replace("\"", "&quot;")
                .replace("'", "&#39;");
}
```

7.5. Desplegar y probar el servlet

1. Guarda ContadorServlet.java.

2. Comprueba que el proyecto no tiene errores de compilación.

3. En Servers, verifica que Decroly está añadido a Tomcat.

4. Si el servidor ya estaba arrancado, usa Publish o reinícialo si Eclipse no publica los cambios.

5. Abre **http://localhost:8080/Decroly/contador**. Debe aparecer el formulario.